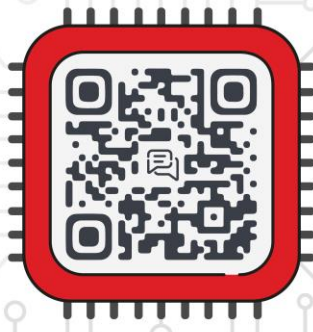




5

17

2550 sp

نظري

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 16/12/2025

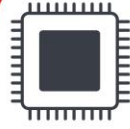
كلية الهندسة المعلوماتية

السنة الأولى

تمثيل الأعداد

د. خولة العلي

01001



#6E2F

مبادئ عمل الحاسوب

بسم الله الرحمن الرحيم

تمثيل الأعداد الصحيحة في النظام الثنائي

1. عدد الخانات الثنائية (Bits)

يُستخدم في تمثيل الأعداد الصحيحة مجموعات من الخانات الثنائية أي تحتوي على 0 أو 1 (Bits)، وغالباً ما تكون من

مضاعفات العدد 8، **مثال:**

8بت، 16بت، 32بت، 64بت

إضافة:

تُستخدم المسجلات (Registers) في المعالجات لتخزين البيانات مؤقتاً أثناء تنفيذ التعليمات، ويُعدّ 512بت أكبر عرض متاح للمسجلات في بعض المعالجات الحديثة، مما يتيح معالجة كميات كبيرة من البيانات بكفاءة عالية

2. عدد القيم الممكنة

عدد الأعداد الصحيحة التي يمكن تمثيلها باستخدام (n) خانة ثنائية هو 2^n

مثال: باستخدام 16 خانة ثنائية يمكن تمثيل

$2^{16} = 65536$ عدداً صحيحاً بدون إشارة (أي كلها موجبة)

التمثيل مع الإشارة (Signed Representation)

طريقة التمييز بين الموجب والسالب

تُستخدم الخانة الأخيرة العليا (الأيسر) لتمثيل الإشارة:

إذا كانت قيمتها 0: العدد موجب

إذا كانت قيمتها 1: العدد سالب



نطاق التمثيل:

عند استخدام 16 خانة:

الأعداد الموجبة المحصورة بين +1 و $2^{16-1} - 1 = 32767$

الأعداد السالبة المحصورة بين -1 و $1 - 2^{16-1} = -32767$

إذاً النطاق الكامل هو من -32768 إلى 32767

■ شرح:

(1 - 16) الرقم الذي طرحته من ال 16 هو العدد (الخانة) الأيسر الذي يعبر عن العدد الموجب والسالب

تمثيل الصفر

في هذا النظام يمكن تمثيل الصفر بطريقتين:

0000000000000000 إشارة موجبة

1000000000000000 إشارة سالبة

ملاحظة:

عند تنفيذ عمليات حسابية مثل الجمع أو الضرب على عددين ممثلين بالنظام الثنائي، قد تتجاوز النتيجة الحد الأقصى للقيم التي يمكن تمثيلها بعدد الخانات الثنائية المتاحة. هذا التجاوز يؤدي إلى تغيير خانة الإشارة، مما ينتج عنه قيمة خاطئة. ولتفادي هذه المشكلة، يُفضل استخدام طريقة تمثيل مختلفة للأعداد السالبة تضمن دقة النتائج، مثل تمثيل المكمل الثاني.

طريقة المكمل أو المتمم مهمة لامتحان :**تمثيل الأعداد السالبة في النظام الثنائي**

مبدأ الإتمام (Complementation)

يستخدم الحاسوب نظرية الإتمام للتعبير عن الأعداد السالبة

الهدف من ذلك هو تحويل عمليات الطرح إلى عمليات جمع لأعداد سالبة، مما يسهل تنفيذ العمليات الحسابية داخل المعالج

بما أن الأعداد ممثلة بالنظام الثنائي، فإن نوعي الإتمام المستخدم هما:

■ الإتمام إلى الواحد (المتمم الأحادي)

■ الإتمام إلى الاثنين (المتمم الثنائي)

الإتمام رياضياً يُكافئ عملية الطرح في أي نظام عدّ



مقارنة بين المتمم الأحادي والثنائي:

النوع	الطريقة الحسابية	النتيجة الرياضية إذا جمعنا العدد (A) مع متممه (B)
المتمم الأحادي (One's Complement)	نعكس كل البتات فقط ($1 \rightleftharpoons 0$)	$A+B = 2^N - 1$
المتمم الثنائي (Two's Complement)	نعكس كل البتات ثم نضيف 1	$A+B = 2^N$

(حيث N تمثل عدد خانات العدد)

أمثلة:

المتمم الأحادي (One's complement):

$(10110010)_2$
 $(01001101)_2$
 المتمم الأحادي

الآن لو جمعنا العدد الأول مع متممه سنحصل على

$$2^n - 1 = 2^8 - 1 = (11111111)_2 = 255$$

المتمم الثنائي (Two's complement):

$(10110010)_2$
 $(01001101)_2$
 + 1
 01001110
 المتمم الثنائي

الآن إذا جمعنا العدد مع متممه سنحصل على

$$2^n = 2^8 = 256 = (10000000)_2$$

الإتمام في النظام العشري

أنواع الإتمام في النظام العشري:

الإتمام إلى التسعة (المتمم إلى 9)

يُحسب بعكس كل رقم إلى ما يكمله ليصبح 9

العلاقة الرياضية $[A + B = 10^n - 1]$ حيث:

(A): العدد الأصلي

(B): المتمم إلى 9

(n): عدد الخانات

الإتمام إلى العشرة (المتمم إلى 10)

يُحسب بعكس كل رقم كما في الإتمام إلى 9، ثم إضافة 1

العلاقة الرياضية $[A + B = 10^n]$

أمثلة توضيحية:

العدد (A)	(B) المتمم إلى 9	المتمم إلى 10
1	8	9
4	5	6
34	65	66
101	898	899

الفرق باختصار:

النوع	طريقة الحساب	النتيجة عند الجمع
المتمم إلى 9	عكس كل رقم ليكمل إلى 9	$10^N - 1$
المتمم إلى 10	عكس كل رقم ثم إضافة 1	10^N

استخدام الإتمام بالحاسوب

استخدام الإتمام الثنائي في الطرح بالحاسوب
الحاسوب لا يُجري عملية الطرح مباشرة، بل يحولها إلى عملية جمع باستخدام المتمم الثنائي للعدد المطروح

خطوات تنفيذ الطرح:

(1) تمثيل العددين بالنظام الثنائي بعدد خانات ثابت (مثلاً 16 خانة)

(2) نحسب المتمم الثنائي للعدد المطروح:

نعكس كل بت (1 $\rightarrow$ 0)

نضيف 1

(3) نجمع العدد المطروح منه مع المتمم الثنائي للمطروح

(4) نهمل الفائض (الخانة الزائدة) الناتجة عن الجمع

مثال:

$$(101)_2 - (111)_2$$

الخطوات:

المتمم الثنائي لـ $(101)_2$ هو:

عكس البتات 010

$$010 + 1 = 011$$

إضافة 1:

$$111 + 011 = 1010$$

نهمل الخانة الزائدة (البت الرابع):

$$010 = (10)_2$$

النتيجة النهائية 10 وهي الفرق الصحيح

ملاحظة: ممكن أن يأتي عدد الخانات المذكوراً بنص السؤال أو نأخذ عدد خانات العدد المعطى

وبشكل آخر: (مع 16 خانة)

0000000000000011
-00000000000000101
0000000000000011
+ 111111111111011
10000000000000010

فائض يهمل

أمثلة:

تمثيل العدد الثماني السالب $(-6372)_8$ بالنظام الثنائي باستخدام المتمم الثنائي:

خطوات:

(1) إيجاد التمثيل الثنائي للقيمة المطلقة للعدد المعطى

$$110\ 011\ 111\ 010_2 = 6372_8$$

(2) نستخدم 16 خانة (بت) ← نضيف أصفاراً يساراً حتى يصبح الطول 16 خانة

$$0000\ 110\ 011\ 111\ 010$$

(3) إيجاد المتمم الثنائي

▪ نعكس كل بت:

▪ نعكس الخانات $(1111\ 001\ 100\ 000\ 101)$

▪ نضيف واحداً إلى الناتج نحصل على العدد المطلوب

$$(-6372)_8 = 1111\ 001\ 100\ 000\ 110_2$$

هذا هو التمثيل الثنائي للعدد $(-6372)_8$ باستخدام المتمم الثنائي على 16 خانة

التحقق:

إذا جمعنا هذا التمثيل مع تمثيل $(+6372)$ نحصل على

$$1\ 0000\ 0000\ 0000\ 0000$$

الناتج يحتوي على فائض (بت زائد) ← يُهمل، وتبقى النتيجة صفر دائماً بعد إهمال البت الزائد

طريقة سريعة لتحويل العدد من النظام الثماني إلى النظام الثنائي:

نجد أن تمثيل أكبر رقم في النظام الثماني (وهو الرقم 7) يحتاج إلى ثلاث خانات في النظام الثنائي حيث: $(7)_8 = (111)_2$

أولاً: نقوم بتجزئة الرقم ولناخذ الرقم السابق على سبيل المثال: 6372 ←
 ثانياً: نقوم بتحويل كل خانة على حدى إلى النظام الثنائي باستخدام 3 خانات: 110 011 111 010

أخيراً: نقوم بدمجهم بجانب بعضهم فنحصل على العدد المراد تحويله إلى النظام الثنائي: $(6372)_8 = (110\ 011\ 111\ 010)_2$

ملاحظة: يمكن بنفس الطريقة تحويل العدد من النظام الست عشري (Hex) إلى الثنائي ولكن باستخدام أربع خانات لأن

أكبر عدد في النظام الست عشري هو الحرف F ويساوي العدد 15 الذي يحتاج أربع خانات في النظام الثنائي: $(1111)_2 = (F)_{16}$

تمثيل الأعداد الكسرية في النظام الثنائي

هناك طريقتين طريقة الفاصلة الثابتة والفاصلة العائمة

▪ الفكرة العامة (طريقة سيغما):

يتكوّن العدد الكسري من جزأين:

الجزء الصحيح ← يُمثّل كما شرحنا سابقاً

الجزء الكسري ← يُمثّل باستخدام قاعدة الأساس a

الجزء الكسري في النظام العشري (للمقارنة فقط):

عند كتابة العدد 0.123 مثلاً:

$$0.123 = 1 \cdot 10^{-1} + 2 \cdot 10^{-2} + 3 \cdot 10^{-3}$$

الجزء الكسري في النظام الثنائي:

يُكتب بالشكل:

$$b = 0.b_1b_2b_3.....b_m$$

ويُحسب باستخدام:

$$b = \sum_{i=1}^m b_i \cdot a^{-i}$$

حيث i تدل على ترتيب الرقم b_i الموجود إلى يمين الفاصلة و a أساس نظام العد
مثال:

المقصود بـ $(0.1)_2$:

$$0.1_2 = 1 \cdot 2^{-1} = (0.5)_{10}$$

باختصار:

النظام	طريقة التمثيل	الأساس المستخدم a
عشري	10^{-i}	10
ثنائي	2^{-i}	2

أمثلة إضافية:

المثال الأول: العدد 0.101_2

الخانات بعد الفاصلة: $b_1 = 1, b_2 = 0, b_3 = 1$

$$\begin{aligned} 0.101_2 &= (1 \cdot 2^{-1}) + (0 \cdot 2^{-2}) + (1 \cdot 2^{-3}) \\ &= 0.5 + 0 + 0.125 = 0.625_{10} \end{aligned}$$

المثال الثاني: العدد 0.011_2

الخانات بعد الفاصلة: $b_1 = 0, b_2 = 1, b_3 = 1$

$$\begin{aligned} 0.011_2 &= (0 \cdot 2^{-1}) + (1 \cdot 2^{-2}) + (1 \cdot 2^{-3}) \\ &= 0 + 0.25 + 0.125 = 0.375_{10} \end{aligned}$$

■ طريقة الفاصلة الثابتة (Fix point)

الفاصلة الثابتة هي طريقة لتمثيل الأعداد الكسرية (التي تحتوي على جزء عشري) داخل الحاسوب باستخدام عدد محدد من الخانات الثنائية (Bits)، بحيث يتم تثبيت موقع الفاصلة مسبقاً ولا يتم تخزينها فعلياً
الفاصلة نفسها لا تُخزن، بل يُفترض موقعها

كيف يتم التمثيل؟

تقسيم الخانات:

يتم تقسيم عدد الخانات الكلي إلى ثلاثة أجزاء:

خانة الإشارة، القسم الصحيح، القسم الكسري

القسم الصحيح: يمثل الجزء **قبل** الفاصلة العشرية (على **يمين** الفاصلة)

القسم الكسري: يمثل الجزء **بعد** الفاصلة العشرية (على **يسار** الفاصلة)

خانة الإشارة: تحدد ما إذا كان العدد موجباً أو سالباً، وتوضع عادة في أقصى اليسار

مثلاً: إذا استخدمنا 16 خانة، يمكن تخصيص 8 خانات للجزء الصحيح و8 خانات للجزء الكسري، وتكون الفاصلة ثابتة

بينهما

موقع الفاصلة: الفاصلة لا تُخزن فعلياً، بل يُفترض أنها موجودة في موقع معين

مثلاً: إذا كانت الفاصلة بعد الخانة الثامنة، فهذا يعني أن أول 8 بتات تمثل الجزء الصحيح، والـ 8 التالية تمثل الجزء الكسري

أمثلة توضيحية:

العدد	تمثيل الجزء الكسري	خانة الإشارة
+0.5	1000000	0
-0.5	1000000	1

نلاحظ أن القيمة 1000000 تمثل 0.5، وخانة الإشارة تحدد ما إذا كانت موجبة أو سالبة

مثال:

يمكن دمج تمثيل القسم الصحيح والكسري من الأعداد الكسرية في عدد من الخانات وتوضع إشارة العدد في الخانة الأخيرة ويكون تقسيم منطقة التخزين كالآتي:

القسم الكسري	القسم الصحيح	الإشارة
--------------	--------------	---------

العدد 1.5

العدد في النظام العشري	التمثيل بواسطة الفاصلة الثابتة
1.5	000000001100000000
-1,5	1111110100000000

العدد 1.5: (+1 +0.5 = +1.5)

التمثيل الثنائي: 000000001 10000000

التفسير:

00000001 = 1 (الجزء الصحيح)

10000000 = 0.5 (الجزء الكسري)

الفاصلة بين الجزأين

الإشارة: موجبة (خانة الإشارة = 0)

العدد -1.5: (-2 +0.5 = -1.5)

التمثيل الثنائي: 10000000 11111110

التفسير:

11111110 = -2 (بالمكمل الثنائي)

10000000 = 0.5

الإشارة: سالبة (خانة الإشارة = 1)



طريقة الفاصلة العائمة

الفاصلة العائمة هي طريقة لتمثيل الأعداد الحقيقية (الكسرية والكبيرة جداً أو الصغيرة جداً) داخل الحاسوب بدقة عالية، وتستخدم عندما لا تكفي الفاصلة الثابتة لتمثيل مدى واسع من القيم

الصيغة الرياضية:

يُكتب العدد بالشكل:

$$X = \pm M \cdot B^E$$

حيث:

M هو الجزء الأعظمي (Magnitude) من العدد ويسمى بالجزء العشري (Mantissa)

B القاعدة (base) المستخدمة في التمثيل

E هو الجزء الأسّي (Exponent)

بعض الأمثلة (b = 10):

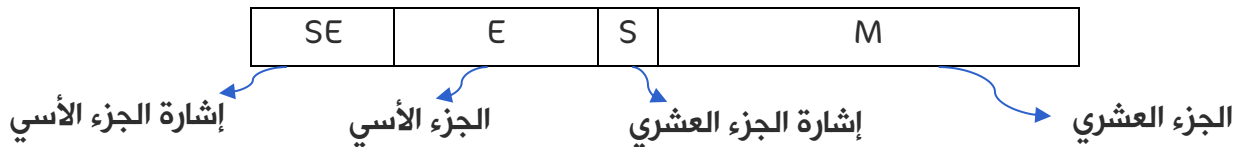
$$21 \cdot 10^{-3} = 0.021$$

$$-21 \cdot 10^3 = -21000$$

$$21 \cdot 10^3 = 21000$$

$$-21 \cdot 10^3 = -21000$$

كيف يتم التمثيل داخل الذاكرة؟



مثال:

عملية تعبير للعدد $125 \cdot 10^6$ باستخدام 8 خانات (6 خانات للجزء العشري و2 للجزء الأسّي)

الجزء العشري						الأسّي	
0	0	0	1	2	5	0	6
0	0	1	2	5	0	0	5
0	1	2	5	0	0	0	4
1	2	5	0	0	0	0	3

ملاحظات مهمة:

أولاً: مشكلة تمثيل العدد صفر في الفاصلة العائمة

عند استخدام طريقة الفاصلة العائمة لتمثيل الأعداد، نواجه صعوبة خاصة في تمثيل العدد صفر، وذلك بسبب طريقة توزيع الخانات بين الجزء العشري والجزء الأسّي

متى تحدث المشكلة؟

عندما يكون الجزء العشري (M = 0)، فإن الناتج الرياضي يكون $B^E = 0$. مهما كانت قيمة الأس (E)، لأن أي عدد مضروب بالصفر يعطي صفرًا

لكن في التمثيل الثنائي داخل الحاسوب، قد تُسند قيمة للأس (E)، حتى عندما يكون (M = 0)، مما يؤدي إلى تمثيل غير دقيق للعدد صفر

هذا يجعل الحاسوب يعتبر العدد صفرًا رغم أن التمثيل قد لا يكون صفرًا رياضياً تماماً، بل قريب جداً من الصفر

ثانياً: قبول الخطأ في التمثيل

في هذه الحالة، يُقبل وجود خطأ بسيط في التمثيل، ويُعتبر العدد صفراً تقريبياً وليس صفراً مطلقاً هذا الخطأ مقبول في التطبيقات التي لا تتطلب دقة مطلقة، مثل الرسوميات أو الحسابات التقريبية

ثالثاً: حدود التمثيل باستخدام عدد محدود من البتات

عند استخدام عدد محدد من البتات لتمثيل الأس، مثل (K) بت، فإننا نستطيع تمثيل الأعداد ضمن مجال معين فقط المجال الممكن للأس: من $2^{(K-1)}$ إلى $2^{(K-1)} - 1$

الانتقال من نظام عد إلى نظام عد آخر**(باستخدام الجدول)****الفكرة:**

يمكن تحويل الأعداد بين الأنظمة (مثل العشري والثنائي) باستخدام جداول تحليل جاهزة، بحيث يُقابل كل رقم عشري تمثيل ثنائي حسب موقعه في العدد

الطريقة:

لكل رقم عشري (من 0 إلى 9)، يوجد تمثيل ثنائي خاص به حسب موقعه:

مثلاً:

الرقم 5 في منزلة الآحاد ← 11110

الرقم 4 في الآحاد ← 11000

الرقم 7 في الآحاد ← 110100

ملاحظات:

الجدول تستخدم لتسهيل التحويل اليدوي دون الحاجة للحسابات المتكررة كل خانة (آحاد، عشرات، مئات، آلاف) لها جدول خاص بها حسب النظام المستهدف

الجدول:

الرمز	الآحاد	العشرات	المئات	الألوف
0	0	0	0	0
1	1	1010	1100100	1111101000
2	10	10100	11001000	11111010000
3	11	11110	100101100	101110111000
4	100	101000	110010000	111110100000
5	101	110010	111110100	1001110001000
6	110	111100	1001011000	1011101110000
7	111	1000110	1010111100	1101001011000
8	1000	1010000	1100100000	1111101000000
9	1001	1011010	1110000100	10001100101000

التحويل من النظام العشري إلى الثنائي

الفكرة:

لتحويل عدد عشري (مثل 4750) إلى ثنائي، نقوم بتحليل كل رقم حسب موقعه (آحاد، عشرات، مئات، آلاف)، ثم نحصل على التمثيل الثنائي لكل جزء ونجمعهم للحصول على الناتج النهائي

خطوات المثال:

العدد 4750

التحليل حسب المنازل:

الرمز	موقعه	التمثيل الموافق
0	آحاد	0
5	عشرات	110010
7	مئات	1010111100
4	ألف	111110100000

1001010001110

بدمج التمثيلات:

$$4750_{10} = 1001010001110_2$$

القسمة المتتالية

هذه الطريقة عامة وبسيطة لتحويل عدد من النظام العشري إلى نظام ذي أساس (b) مثل الثنائي (b = 2) أو الثماني (b = 8)

الفكرة: نقسم العدد على الأساس مراراً ونحتفظ بالبواقي ثم نقرأ البواقي من الأسفل إلى الأعلى

خطوات القسمة المتتالية:

- نقسم العدد الصحيح على (الأساس)
- نضع الباقي ونضع ناتج القسمة في خانة المقسوم
- نكرر 1 و 2 حتى يصبح العدد المقسوم يساوي الصفر
- ننظر إلى خانة الباقي من الأسفل إلى الأعلى ونسجل الأرقام التي نراها من اليسار إلى اليمين

أمثلة:

$$(160)_{10} = (?)_8$$

الباقي	أساس	المقسوم
1	8	169
5	8	21
2	8	2
0		0

→ (251)₈

$$(13)_{10} = (?)_2$$

الباقي	أساس	المقسوم
1	2	13
0	2	6
1	2	3
1	2	1
0		0

→ (1101)₂

الترميز العشري المرمز ثنائياً BCD

Binary Coded Decimal

التعريف:

الـ BCD هو نظام يُستخدم لتمثيل الأعداد العشرية داخل الحاسوب باستخدام الترميز الثنائي، بحيث يُمثل كل رقم عشري (من 0 إلى 9) باستخدام 4 خانات ثنائية

مثال:

العدد العشري 725 يُمثل في BCD كالتالي:

$0111 \leftarrow 7$

$0010 \leftarrow 2$

$0101 \leftarrow 5$

الناتج 0111 0010 0101

خطوات تحويل عدد عشري إلى BCD:

1. فصل الأرقام العشرية:

مثلاً: $725 \leftarrow 7, 2, 5$

2. تحويل كل رقم إلى ثنائي مكون من 4 بتات :

$0111 \leftarrow 7$

$0010 \leftarrow 2$

$0101 \leftarrow 5$

3. ترتيب التمثيلات جنباً إلى جنب:

$$725_{10} = 011100100101_{BCD}$$

خطوات تحويل عدد من BCD إلى عشري:

1. قسّم العدد الثنائي إلى مجموعات من 4 بتات بدءاً من اليمين إلى اليسار

2. أضف أصفاراً إلى اليسار إذا كان عدد الخانات أقل من 4

3. حوّل كل مجموعة إلى رقم عشري

4. اقرأ الأرقام من اليمين إلى اليسار

مثال: $BCD (10000010110)$ نضيف 0 إلى يسار العدد الثنائي الأخير ليصبح 0100

0100	0001	0110
4	1	6

$(416)_{10} \leftarrow$



fear kills more dreams
than failure ever will

العمليات الحسابية داخل BCD

عند إجراء عمليات مثل الجمع داخل نظام BCD، يجب الانتباه إلى أن:
كل رقم عشري يُمثل بأربع خانات ثنائية
عند الجمع، إذا تجاوز الناتج قيمة 9 أي (1001)، فإن الناتج لا يكون صالحاً في BCD ويجب تصحيحه
كيف نُصحّح الناتج؟

نضيف قيمة 0110 (أي 6 عشرياً) إلى الناتج غير الصالح
هذا يُعيد التمثيل إلى نطاق BCD الصحيح

مثال:

إجراء العملية الحسابية بين عددين مرمزين بالترميز العشري المرمز ثنائياً:

$$55 + 99$$

ناتج جمع 5 5

9 9 و

هو 1 5 4

الترميز الثنائي للنصوص

الفكرة العامة:

عند تمثيل النصوص داخل الحاسوب يتم تحويل كل حرف (Character) إلى رمز ثنائي خاص به، وذلك باستخدام جداول ترميز مثل ASCII أو غيره

تصنيف المحارف:

باستخدام جدول التطابق

B. أرقام Digital
0
1
2
⋮
9

مثال:
3 = 00000011
8 = 00001000

A. محارف Alphabet
A
B
C
⋮
Z

مثال:
A: 01101101
B: 10001010
M: 01001101

C. رموز Symbols
+
,
\$

&
[]

D. تحكم Control
.
,
⋮
;

وحدات التخزين والتمثيل الثنائي

الأساس:

اللغة الأساسية في الحاسوب هي الثنائية (0 و1)، وكل البيانات تُخزن بهذه الطريقة
العلاقة بين البت والبايت:

$$1 \text{ Byte} = 8 \text{ Bits}$$

كل حرف (Character) يُخزن باستخدام 1 Byte

■ وحدات قياس التخزين:

عند تخزين المعطيات بالحاسوب:

$$\text{Kilo Byte} = 1024 \text{ Byte} = 2^{10}$$

$$\text{Mega Byte} = 1024 \text{ Kilo Byte} = 2^{20}$$

$$\text{Gega Byte} = 1024 \text{ Mega Byte} = 2^{30}$$

$$\text{Terra Byte} = 1024 \text{ Gega Byte} = 2^{40}$$

■ مسألة (1):

بفرض صفحة كتاب فيها 200 كلمة وكل كلمة وسطياً فيها 5 محارف:

(1) ما هو عدد المحارف؟

(2) ما هي مساحة التخزين لهذه الصفحة (علماً أن المحرف يأخذ مساحة 1 byte)؟

■ الحل:

$$\text{محرف} \times 200 = 1000 \quad 5 \times$$

$$1000 \text{ byte} \rightarrow 1 \text{ KB} \quad (\text{تقريباً})$$

ترميز التعليمات (Instruction Encoding)

الفكرة:

كل تعليمات تُنفَّذ داخل المعالج تتكوّن من جزأين:

■ العملية المطلوب تنفيذها:

مثل: الجمع، النقل، المقارنة...

■ المعطيات التي تُنفَّذ عليها العملية:

غالباً تكون عنواناً في الذاكرة أو موقعاً في ملف

كل عملية تُرمّز برقم خاص، وكل عنوان يُمثّل أيضاً برقم ثنائي، ليتمكن المعالج من فهمها وتنفيذها

ترميز الحروف باستخدام ASCII وغيره

ما هو الترميز؟

الترميز هو عملية إعطاء رقم ثنائي لكل حرف أو رمز، ليتمكن الحاسوب من تخزينه ومعالجته

النظام	الاستخدام
ASCII	الحروف اللاتينية والرموز العامة
ASMO	الحروف العربية (منظمة المقاييس العربية)
EBCDIC	الحروف اللاتينية (IBM)

كيف يعمل الترميز؟

كل حرف يُعطى رقمًا ثنائيًا مكونًا من 7 أو 8 بتات

عند الإدخال: تحويل الحروف إلى أرقام

عند الإخراج: تحويل الأرقام إلى حروف ليقرأها المستخدم

جدول ترميز المحارف وفق معيار ASCII

المحرف	الرمز العشري	المحرف	الرمز العشري	المحرف	الرمز العشري	المحرف	الرمز العشري
a	97	A	65	!	33	NUL	1
b	98	B	66	"	34	SOH	2
c	99	C	67	#	35	STX	3
d	100	D	68	\$	36	ETX	4
e	101	E	69	%	37	ENQ	5
f	102	F	70	&	38	ACK	6
g	103	G	71	'	39	BEL	7
h	104	H	72	(	40	BS	8
i	105	I	73	)	41	HT	9
j	106	J	74	*	42	LF	10
k	107	K	75	+	43	VT	11
l	108	L	76	,	44	FF	12
m	109	M	77	_	45	CR	13
n	110	N	78	.	46	SO	14
o	111	O	79	/	47	DI	15
p	112	P	80	0	48	DLE	16
q	113	Q	81	1	49	DC1	17
r	114	R	82	2	50	DC2	18
s	115	S	83	3	51	DC3	19
t	116	T	84	4	52	DC4	20
u	117	U	85	5	53	NAK	21
v	118	V	86	6	54	SYN	22
w	119	W	87	7	55	ETB	23
x	120	X	88	8	56	CAN	24
y	121	Y	89	9	57	EM	25
z	122	Z	90	:	58	SUB	26
{	123	[	91	;	59	ESC	27
	124	\	92	<	60	FS	28
}	125	]	93	=	61	GS	29
~	126	^	94	>	62	RS	30
DEL	127	_	95	?	63	US	31
			96	@	64	SP	32
محارف تحكم							

ترميز الصور

الفكرة الأساسية:

تمثيل الصور رقمياً داخل الحاسوب يتم عبر تقسيم الصورة إلى عناصر صغيرة تسمى بيكسلات (Pixels)، وكل بيكسل يُعبر عن نقطة ضوئية لها لون أو درجة إضاءة معينة

خطوات التمثيل الرقمي للصورة:

1. التقسيم إلى بيكسلات:

تقسّم الصورة إلى شبكة من الصفوف والأعمدة، وكل تقاطع يمثل بيكسل واحد

2. تحديد دقة الصورة (Resolution):

تقاس بعدد النقاط في البوصة الواحدة (DPI = Dots Per Inch)

مثلاً: صورة بدقة 300 x 300 DPI تعني 300 نقطة أفقية x 300 نقطة عمودية في كل بوصة (1 بوصة = 2.54 سم)

3. التحويل الرقمي (Digitizing):

يتم تحويل كل بيكسل إلى قيمة رقمية تمثل درجة الإضاءة أو اللون، لأن الحاسوب لا يفهم إلا الأرقام الثنائية

4. التمثيل في الذاكرة:

تُخزن الصورة في ملف يُسمى Bitmap، وهو عبارة عن مصفوفة من القيم الرقمية تمثل صفوف وأعمدة البيكسلات

ملاحظات مهمة:

- كلما زاد عدد البيكسلات، زادت دقة الصورة
- كل بيكسل يُخزن على شكل رقم ثنائي يمثل لونه أو شدته
- Bitmap هو الشكل الأساسي لتخزين الصور في الذاكرة

تدقيق المعلومات لحمايتها من الأخطاء

عند إرسال البيانات رقمية من جهاز إلى آخر، قد تحدث أخطاء أثناء النقل، لذلك نستخدم بتات التدقيق (Parity Bits) لاكتشاف هذه الأخطاء

أنواع التدقيق:

- التدقيق الزوجي (Even Parity):
نضيف بتاً بحيث يكون عدد الـ 1 في السلسلة الثنائية زوجياً
- التدقيق الفردي (Odd Parity):
نضيف بتاً بحيث يكون عدد الـ 1 فردياً



مثال من الجدول:

نفترض أن المعلومات المرسله من حاسوب إلى آخر هي الأحرف A, B, C, D يقوم الحاسوب بإرسال هذه السلسلة إضافة إلى حرف التدقيق في نهاية السلسلة. يبين الجدول الرموز المعطاة السابقة باعتماد نظام التدقيق الزوجي:

الحرف	الرمز الثنائي	خانة التدقيق
A	1000001	0
B	1000010	0
C	1000011	1
D	1000100	0
حرف التدقيق	0000100	1

في حالة الحرف C مثلاً، يحتوي الرمز الثنائي على عدد فردي من الـ 1، لذا نضيف 1 ليصبح العدد زوجياً ويصبح 10000111
 ← إذاً يساعد هذا النظام في اكتشاف الأخطاء البسيطة أثناء نقل البيانات، مما يزيد من موثوقية الاتصال بين الأجهزة حيث ينتظر المستقبل عدداً فردياً أو زوجياً من الـ 1 في كل بايت وأي خطأ يمكن ملاحظته بسهولة

ضغط المعلومات (Data Compression)لماذا نحتاج إلى ضغط المعلومات؟

في عصر البيانات الضخمة، أصبح حجم المعلومات التي نتعامل معها كبيراً جداً، مما يتطلب:

- مساحات تخزين ضخمة
- زمن أطول لنقل ومعالجة البيانات
- لذلك، ظهرت الحاجة إلى تقنيات ضغط المعلومات بهدف:
- تقليل حجم البيانات
- نقل أكبر كمية منها في وقت أقل

كيف تعمل تقنيات الضغط؟

تعتمد تقنيات الضغط على مبدأ وجود ارتباطات وتكرارات بين الرموز أو الكلمات داخل البيانات، ويتم ذلك من خلال:

1. الدراسة الإحصائية:

تحليل تواتر (تكرار) مكونات المعلومات المخزنة أو المتبادلة، مثل:

الكلمات المتكررة

التركييب اللغوية (جمل، صيغ، مصطلحات)

التلازمات (مثل "الكتاب"، "في هذا السياق"، إلخ)

2. الترميز الرمزي:

استبدال العناصر المتكررة برموز مختصرة، مما يقلل عدد الخانات المطلوبة للتخزين

مثال:

- تقليل حجم التخزين
- تقليل زمن المعالجة والنقل

فمثلاً: لضغط النصوص يمكن دراسة تواتر الأحرف، وتواتر الثنائيات (حرفان متعاقبان)، والثلاثيات، الخ وإعطاء رموز لهذه التراكيب. ففي اللغة العربية تتكرر الثنائية "ال" عدداً كبيراً من المرات في النصوص. فإذا كان لدينا نص يحوي 10000 حرفاً وتكررت الثنائية "ال" 500 مرة فإن إعطاء رمز واحد لهذه الثنائية يعني اختصار 500 ثمانية

الفائدة النهائية:

بعض تقنيات الضغط تستطيع تقليص حجم البيانات إلى أقل من 10% من حجمها الأصلي، مما يحقق:

- كفاءة عالية في التخزين
- سرعة أكبر في المعالجة
- توفير في الموارد التقنية

^_^

